МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное автономное образовательное учреждение высшего образования «Санкт-Петербургский политехнический университет Петра Великого»

Институт компьютерных наук и кибербезопасности

Высшая школа технологий и искусственного интеллекта

Направление: 02.03.01 Математика и компьютерные науки

Отчет по практическим работам по дисциплине «Функциональное программирование»

Студент,
5130201/40001 _____ Анари А.

Преподаватель,
к. т. н. _____ Моторин Д. Е.

«_____»_____ 2026 г.

Санкт-Петербург, 2026

Содержание

Введение **3**

1 График выполнения и сдачи работ **4**

2 Описание коммитов **6**

3 Защита практических работ **11**

3.1 Практическая работа 1 11
3.2 Практическая работа 2 11
3.3 Практическая работа 3 11
3.4 Практическая работа 4 11
3.5 Практическая работа 5 12

4 Структура репозитория **13**

Приложения **14**

Введение

Данный отчёт описывает процесс выполнения и защиты практических работ по дисциплине «Функциональное программирование». В ходе семестра было выполнено пять практических заданий. Отчёт отражает хронологию работы, использованные инструменты и результаты защиты.

Общий подход и порядок работы для выполнения практических заданий:

Чтобы подготовиться к каждому заданию, я сначала читал файлы с лекциями и начинал самостоятельно выполнять примеры и задания из них. Я использовал ИИ, чтобы он давал мне простые практические задания после каждого раздела лекций, и выполнял эти задания в отдельных файлах. В основном я пользовался Claude Sonnet 4.6, и мои промпты были примерно такими: «Объясни этот раздел лекции, а затем дай мне 3–5 дополнительных практических заданий по программированию, которые начинаются с лёгкого уровня и постепенно усложняются, чтобы убедиться в понимании материала лекции». Затем я сверял свои ответы с ИИ и задавал ему вопросы, чтобы убедиться, что я понял лекции.

Обычно у меня уходило около 3–6 часов на полное завершение одной лекции, а после каждой лекции я писал практическое задание, прилагаемое к ней, полностью самостоятельно, без использования ИИ. Итак, резюмируя: я использовал ИИ для перевода, понимания и отработки лекционного материала, но не для выполнении практических заданий. После того как я заканчивал с практическими заданиями, вечером я коммитил свои файлы с самостоятельной практикой и официальные практические задания в репозиторий.

Так проходил каждый четверг с самого начала семестра, без единого исключения. Это был мой общий подход и порядок работы при изучении лекций и выполнении практических заданий.

1 График выполнения и сдачи работ

В данном разделе представлены временная диаграмма выполнения и сдачи работ (см. рисунок 1), полный журнал коммитов репозитория (см. таблицу 1), а также список задач (issues) с соответствующими им коммитами (см. таблицу 2).

Таблица 1 содержит полный журнал коммитов с датами и сопоставлением каждого коммита с номером практической работы.

Таблица 2 содержит все задачи (issues), зафиксированные в репозитории, с указанием коммита, закрывшего каждую из них.

Таблица 1: Журнал коммитов репозитория

Хэш	**Дата и время**	**Сообщение коммита**	**ПР**
525a3cd	20.02 15:56	first commit, making repo	—
48da3ba	20.02 16:02	ds store remove	1
8da96d8	20.02 16:03	clean up	1
44416c9	20.02 17:27	first part done	1
271b905	20.02 19:19	finished 1st practice	1
229ce01	20.02 19:20	final change	1
89bc067	01.03 18:28	first part of practice01_2.hs	1
8e9d72d	05.03 18:03	finished practice 1	1
475b4e9	06.03 13:54	fixed myfold1	1
c808add	06.03 14:31	todo 1, wrote test cases for cherrycake and fixed myFoldl1	1
87a6118	06.03 14:43	todo1	1
97e6086	06.03 14:56	final commit lab1	1
4853ff4	08.03 21:25	practice 2 done	2
abf1ef7	12.03 01:16	header cleanup	2
ffa32f2	20.03 01:25	finally done with pr02_2.hs	2
92adb2d	20.03 08:04	clean up	2
6a9a55f	20.03 08:26	added second implementation for MyTree	2
66969fb	20.03 08:42	debugged, added foldable to MyTree	2
7a77be1	22.03 08:47	practiced lecture materials, added practice 9 & 10	2
013e463	22.03 10:40	MyParser now runs perfectly	2
8c93ccf	22.03 10:44	annoying warnings cleared	2
2575b73	22.03 12:46	finished implementing pr2-3	2
16b984e	22.03 12:52	cleanup	2
61a19e6	26.03 19:04	small review	2
bfcb75d	26.03 19:31	fixed an error in MyMaybe that broke Monoid Law	2
26863a4	26.03 19:47	fixed a wrong example	2
87bb47c	26.03 19:54	clean up	2
2fae5a9	26.03 20:14	fixed imports with () which imports NOTHING	2
cbbb2d6	26.03 23:25	cleanup on MyEither	2
4d4ebb5	26.03 23:28	added self practice files	2
7947932	27.03 08:34	clean up	2
a48631c	02.04 12:28	added glitch project for pr3-1	3
1ff58b9	12.04 13:05	solved pr03_2.hs	3
8c1e60c	16.04 19:34	clean up	3
13a6b8b	17.04 23:36	added Monadpr	4
fb00cfe	19.04 16:35	wrote the transformer stack for practice 4	4
3366b62	19.04 17:54	fixing the dependencies on mtl	4

Таблица 1: Журнал коммитов репозитория

Хэш	**Дата и время**	**Сообщение коммита**	**ПР**
7f14dc8	19.04 17:55	add file logging	4
46307e5	19.04 17:56	added a big maze	4
17c6fc2	30.04 15:51	added Practice5 and QCpr	5
5de415f	01.05 18:51	just added pr04_1.hs and pr05_1.hs	4, 5

Таблица 2: Задачи (issues) репозитория и закрывающие коммиты

#	**ПР**	**Название задачи**	**Закрыта**	**Коммит**
1	4	trying hard to understand Monads	17.04 20:36	13a6b8b — added Monadpr
2	4	removing mtl from build dependencies	19.04 14:54	3366b62 — fixing the dependencies on mtl
3	4	wanted to have the logs in a separate file	19.04 15:07	7f14dc8 — add file logging
4	4	real game	19.04 14:56	46307e5 — added a big maze (Fixes #4)
5	5	Maybe a mistake in the book	30.04 12:59	17c6fc2 — added Practice5 and QCpr

На рисунке 1 представлена временная диаграмма выполнения практических работ: коммиты и даты сдачи отмечены разными цветами. Даты сдачи работ:

- ПР1 — 06.03.2026
- ПР2 — 03.04.2026
- ПР3 — 17.04.2026
- ПР4 — 24.04.2026
- ПР5 — 08.05.2026

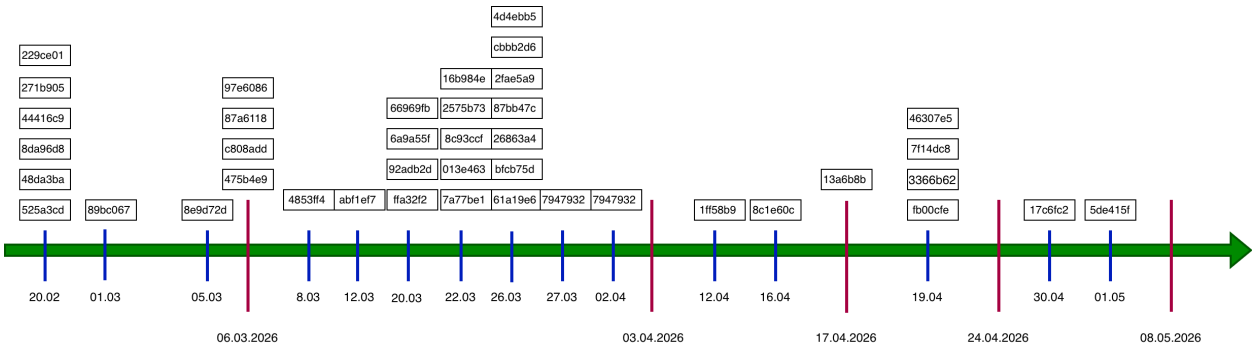

Рис. 1: Временная диаграмма выполнения и сдачи практических работ

2 Описание коммитов

Небольшие коммиты, не требующие отдельного пояснения, объединены в группы. Ряд коммитов описывается отдельно — это крупные коммиты, отражающие работу за несколько дней: как правило, они связаны с изучением объёмных разделов лекций, например монад, где требовалось глубокое погружение в материал.

525a3cd, 48da3ba, 8da96d8 *20.02.2026*

Инициализация рабочего пространства: создание репозитория, настройка `.gitignore` и очистка директории.

- `525a3cd` — создание репозитория, первоначальная структура проекта.
- `48da3ba` — удаление служебных файлов `.DS_Store` из отслеживания.
- `8da96d8` — общая очистка рабочего пространства.

44416c9 *20.02.2026*

Начало выполнения ПР1: реализованы функции первой части `pr01_01`.

- `myFST`, `mySND`, `myTHRD` — извлечение элементов тройного кортежа.
- `myHead`, `myTail` — отделение головы и хвоста списка.
- `myTake`, `myDrop` — взятие и отбрасывание первых n элементов списка.
- `myProduct` — произведение элементов списка.
- `myZip`, `myZip3`, `myUnzip` — объединение и разъединение списков в кортежи.

271b905, 229ce01 *20.02.2026*

Завершение второй части `pr01_01`: реализованы оставшиеся функции высшего порядка.

- `myFilter`, `myFilter2` — фильтрация списка (два варианта: через охранные выражения и через `if-else`).
- `myMap` — применение функции к каждому элементу списка.
- `myzipWith`, `myZipWith3` — попарное и потройное применение функции к спискам.
- `myAll`, `myAll2`, `myAny`, `myAny2` — проверка предиката на всех/хотя бы одном элементе (два варианта каждая).
- `myComposition` — композиция двух функций.

89bc067 *01.03.2026*

Начало выполнения ПР1, часть 2: создан файл `pr01_2.hs`. Реализованы функции на основе сверток (`myReverse`, `myTakeWhile`, `mySpan`), свёртки для непустых списков (`myFoldl1`, `myFoldr1`), работа с неравными списками (`myZipSave`, `myUnzipSave`), а также `myMaybe`, `myMap` для пользовательского типа `MyList` и развёртка `myUnfoldr`.

8e9d72d *05.03.2026*

Завершение ПР1: дописана вторая часть `pr01_2.hs`. Реализованы алгебраические типы данных для выпекания тортов

475b4e9, c808add, 87a6118, 97e6086 *06.03.2026*

Исправления по итогам защиты ПР1: устранены замечания, полученные в ходе защиты, — исправлена `myFoldl1` и написаны тестовые случаи для функций выпекания тортов. Вопросы, заданные в ходе защиты, и их решения приведены в разделе 3.

4853ff4, abf1ef7 *08.03 — 12.03.2026*

Выполнена ПР2, часть 1: создан cabal-проект `MyEvolProject`, добавлен модуль `MyEvolModule.hs` с реализацией инстансов классов типов, проект собран и протестирован. Коммит `abf1ef7` — небольшая правка заголовков.

ffa32f2, 92adb2d *20.03.2026*

Крупный коммит по ПР2: предварительно отработан весь лекционный материал в файлах самостоятельной практики (`practice2.hs`–`practice8.hs` в директории `self practice`), затем выполнена ПР2, часть 2 — создан stack-проект `myParser` с модулями `MyMaybe`, `MyEither`, `MyTree`, написаны и прогнаны тестовые случаи.

6a9a55f *20.03.2026*

В `MyTree.hs` добавлена вторая реализация дерева, в которой операция применяется к каждому листу относительно всего дерева.

66969fb *20.03.2026*

Отладка проекта: добавлен инстанс `Foldable` для `MyTree`, обновлён `Main.hs` с тестовыми вызовами, скорректированы зависимости в `package.yaml` и `.cabal`.

7a77be1 *22.03.2026*

Самостоятельная практика по лекционному материалу до раздела `Alternative`: добавлены файлы `practice9.hs` и `practice10.hs`. Отработаны: пользовательский тип `Parser`, инстансы `Functor`, `Applicative` и `Alternative` для него, примитивные парсеры (`satisfy`, `char`, `digit`, `lower`, `upper`), комбинаторы `some`/`many`, построение парсеров выражений (`finalMult`, `plusOrMult`).

013e463, 8c93ccf *22.03.2026*

Реализован модуль `MyParser.hs` в рамках ПР2, часть 3: парсер построен поверх собственного типа `MyMaybe` вместо стандартного `Maybe`. Реализованы инстансы `Functor`, `Applicative`, `Alternative`, примитивные парсеры и комбинаторы (`lowers`, `digits`, `finalMult`, `plusOrMult`). Обновлён `Main.hs`, созданы заглушки для `MyParsec` и `MyAttoparsec`. Коммит `8c93ccf` — устранены предупреждения компилятора.

2575b73, 16b984e *22.03.2026*

Завершена ПР2, часть 3: реализованы `MyParsec.hs` и `MyAttoparsec.hs`. `MyParsec.hs` использует готовые функции из `Text.Parsec` с написанными обёртками для сохранения интерфейса `Main.hs`. `MyAttoparsec.hs` использует функции из `Data.Attoparsec.Text` с применением `pack` из `Data.Text` для передачи строки в `parseOnly`. Также добавлены файлы самостоятельной практики `practice11.hs` и `practice12.hs`. Коммит `16b984e` — финальная очистка.

61a19e6, bfcb75d, 26863a4, 87bb47c, 2fae5a9, cbbb2d6 *26.03.2026*

Серия мелких исправлений по проекту: исправлена ошибка в `MyMaybe`, нарушавшая закон моноида; исправлен некорректный пример, оставшийся с предыдущей версии; устранена ошибка с импортом вида `import M ()` (импортирует ничего); очистка модуля `MyEither`.

4d4ebb5, 7947932 *26.03 — 27.03.2026*

Добавлены файлы самостоятельной практики поздно ночью; в сообщении коммита отмечено, что часть содержимого `practice12.hs` была случайно перезаписана от усталости. `practice12.hs` — практика по классам типов: пользовательский тип `Direction` с инстансами `Show`, `Eq`, `Enum`, `Bounded`; `Foldable` и `Monoid` для `MyEither` и `MyMaybe`. `practice13.hs` — повторная отработка реализации `Parser` поверх `MyMaybe` с инстансами `Functor`, `Applicative`, `Alternative` и парсерами-комбинаторами.

a48631c *02.04.2026*

Крупный коммит: создан stack-проект `glitches` для ПР3, часть 1, добавлены файлы самостоятельной практики `practice14` и `practice15`. Файлы `practice14` и `practice15` — самостоятельная практика по материалам книги *Get Programming with Haskell* Уилла Курта, посвящённой теме глитч-арта.

Проект реализует глитч-арт на изображениях через манипуляцию байтами (`Data.ByteString`). Модуль `Purefuncs.hs` содержит чистые функции: `replaceByte` — замена одного байта по по-

зиции, `sortSection` — сортировка участка байтов с реверсом. `Main.hs` реализует три режима глитча: случайная замена байта, случайная сортировка участка, и их комбинация через `foldM` (вместо цепочки ручных вызовов, как предлагалось в книге) — применяется список из 3 замен и 2 сортировок. Результаты записываются в отдельные файлы; в репозиторий включены выходные изображения (`lovecraft.jpg` и тестовые снимки).

1ff58b9, 8c1e60c *12.04 — 16.04.2026*

Выполнена ПР3, часть 2: создан stack-проект `Route` с реализацией поиска пути в лабиринте через монадный трансформер `RWS`. `MazeSolver.hs` определяет тип `Maze` (`Map String [String]`), монадный стек `Solver` (`RWS Maze [String] (Set String)`), и реализует DFS через взаимно-рекурсивные функции `dfs` и `tryNeighbors` — по аналогии с `some`/`many`. `Main.hs` запускает решатель через `runRWS` на тестовом лабиринте и выводит лог посещений и найденный путь. Также предпринята попытка применить глитч-эффекты к `.bmp`-формату — без успеха; соответствующие файлы включены в коммит.

До выполнения этого задания были отработаны монады `Reader`, `Writer` и `State` в файлах самостоятельной практики, однако они не были закоммичены здесь — все файлы практики по монадам включены в следующий коммит.

13a6b8b *17.04.2026*

Самый крупный коммит в репозитории, закрывает задачу #1. Все файлы самостоятельной практики по монадам и монадным трансформерам, накопленные за несколько дней, зафиксированы в одном коммите в директории `self practice/MonadPr/`. Материал изучался по двум источникам: книга *Get Programming with Haskell* Уилла Курта (главы 30–32) и *Learn Haskell by Example* (глава 15, монадные трансформеры).

Содержимое директории `MonadPr/`:

- `MONADNOTES.hs` — конспект-шпаргалка по API монад `Reader`, `Writer`, `State`: ключевые функции, сигнатуры и примеры для повторения.
- `MReaderpr.hs` — практика монады `Reader`: `ask`, `asks`, `local`, цепочки вычислений в контексте окружения.
- `MWriterpr.hs` — практика монады `Writer`: `tell`, `listen`, `censor`, накопление лога.
- `MStatepr.hs` — практика монады `State`: `get`, `put`, `modify`, цепочки состояний через `replicateM`.
- `MTrpr.hs` — практика монадных трансформеров: стеки `ReaderT`/`WriterT`/`StateT`, `lift`, `runReaderT`/`runWriterT`/`runStateT`.
- `MPTCpr.hs` — практика `MultiParamTypeClasses`: класс `Mult` для умножения векторов и матриц.
- `MRWSpr.hs` — практика монады `RWS` как комбинации `Reader`/`Writer`/`State`.
- `Maze/` — самостоятельный stack-проект: повторная реализация лабиринта через `RWS` для закрепления материала перед сдачей ПР3.

Содержимое директории `getprgwithHaskell/`:

- `chapter30.hs` — глава 30: монада `IO`, do-нотация, `>>=` и `>>`.
- `chapter31.hs` — глава 31: работа с `IO` и модулями, модуль `Pizza`.
- `chapter32.hs` — глава 32: монада списков, `guard`, списочные включения через `do`.
- `pizza.hs` — упражнение из книги: интерактивная программа для сравнения цен на пиццу через `IO`.
- `review.hs` — упражнение на повторение: комбинирование `Maybe`, `Applicative` и функторов.

- `practice16.hs` — дополнительная самостоятельная практика по материалу глав.

Лекционный материал по монадам (лекции 9, 10, 11) отработан в отдельных stack-проектах, по одному файлу на каждый тип монады:

- `lec09/`: `MonadMaybe`, `MonadList`, `MonadReader`, `MonadWriter`, `MonadId`, `MonadExcept`.
- `lec10/`: `MonadState`, `MonadRWS`, `MonadIO`, `MonadLogic`, `MonadPar`.
- `lec11/`: `TrMon` — трансформеры монад; `TrRW` — трансформеры `Reader`/`Writer`; `MPTC` — многопараметрические классы типов.

Стоит отметить, что изучение монад и привыкание к ним с той скоростью, с которой шли лекции, было настоящим испытанием. Я чувствую, что действительно выложился по полной в этой теме — за те две недели, что я практиковался с монадами, было много ночей, когда я засиживался допоздна. Мне кажется, мне удалось их освоить благодаря использованию упомянутых книг и источников. Стоит сказать, что монады стали одной из самых запоминающихся частей моего обучения на данный момент!

fb00cfe *19.04.2026*

Начало ПР4: создан stack-проект `MazeGame`, реализован модуль `MyTransformerStack.hs`. Модуль содержит реализацию трёх монадных трансформеров с нуля: `ReaderT` (с инстансами `Functor`/`Applicative`/`Monad`/`MonadTrans` и функциями `ask`, `asks`, `local`), `WriterT` (с `tell`, `listen`, `censor`) и `MyStateT` (с `get`, `put`, `modify`). Класс `MonadTrans` также определён вручную. Реализации перенесены из файлов самостоятельной практики и адаптированы под нужды проекта. Файл `GameEngine.hs` создан пустым — работа над ним запланирована в следующих коммитах.

3366b62 *19.04.2026* (закрывает задачу #2)

Реализован `GameEngine.hs` и исправлены зависимости. Задача #2 была связана с тем, что `mtl` использовался только ради `runIdentity` — он заменён на `Data.Functor.Identity` из `base`, и `mtl` полностью убран из зависимостей, что соответствует цели задания: трансформерный стек написан вручную. `GameEngine.hs` определяет монадный стек `Game a = MyStateT RoomName (WriterT [String] (ReaderT Maze IO)) a`, функции игрового цикла (`gameTurn`, `gameLoop`), парсер лабиринта из файла (`parseMaze`, `loadMaze`) и вспомогательную функцию `io` для подъёма `IO`-действий через три слоя трансформеров. Добавлен тестовый файл лабиринта `maze.txt`.

7f14dc8, 46307e5 *19.04.2026* (закрывают задачи #3 и #4)

Два завершающих коммита ПР4. `7f14dc8` (задача #3) — добавлен файл `logs.txt` с сохранёнными логами игровых сессий, реализованных через функцию `saveLogs` в `GameEngine.hs`. `46307e5` (задача #4) — добавлен большой лабиринт `maze1.txt` для полноценной игры: 22 комнаты с разветвлённой структурой (главный зал, библиотека, арсенал, подземелье, деревня, таверна, крипта и т.д.) с единственным путём до финала через `throne -> finish`.

17c6fc2 *30.04.2026* (закрывает задачу #5)

Крупный коммит: выполнена ПР5, добавлена самостоятельная практика по QuickCheck в директории `self practice/QCpr/`.

Подготовка к заданию: лекционный материал изучен полностью, просмотрено видео о `classify` и `collect`, прочитаны главы 36 и 37 книги *Get Programming with Haskell* Уилла Курта. В процессе изучения была обнаружена ошибка в коде из книги (задача #5): функция `isPrime` использует охранное выражение `n >= length primes` в качестве верхней границы, что логически неверно. При `primes = sieve [2..20]` длина списка равна 8, поэтому `isPrime 19` возвращает `Nothing`, хотя 19 является простым числом. Исправление: заменить `length primes` на `head . reverse $ primes`, т.е. сравнивать с наибольшим элементом списка, а не с его длиной — именно так реализовано в `QCpr/primes/src/Primes.hs`.

Реализация ПР5 (проект `practice5/prac5/`):

- `src/Lib.hs` — реализованы функции `addMod` и `reverseWords`; здесь же находятся черновые варианты всех property-тестов с подробными комментариями и результатами запусков `quickCheck` — оставлены намеренно как рабочие заметки.
- `test/Spec.hs` — финальная чистовая версия: 7 property-тестов с корректными именами (`prop_addModMatchesModuloAddition`, `prop_addModHasNeutralElement`, `prop_addModIsCommutative`, `prop_reverseWordsEmptyString`, `prop_reverseWordsSingleWord`, `prop_reverseWordsChangesWordOrder`, `prop_reverseWordsTwiceReturnsOriginal`).

Содержимое директории `self practice/QCpr/`:

- `palindrome-testing/` — практика по примеру из книги: тестирование функции `isPalindrome` через QuickCheck.
- `primes/` — практика тестирования `isPrime` и `primeFactors` с исправлением ошибки из книги; реализованы тесты `prop_validPrimesOnly`, `prop_primesArePrime`, `prop_nonPrimesAreComposite`, `prop_factorsMakeOriginal`, `prop_allFactorsPrime`.
- `prac5/` — черновой вариант проекта ПР5, использовался для экспериментов перед написанием финальной версии.
- `pr05_1.hs`, `notes.hs` — вспомогательные файлы с заметками.

3 Защита практических работ

3.1 Практическая работа 1

Вопрос 1. Написать тест для функции `makeCake`.

Ответ:

```
testmake = makeCake (Just CherryDough) (Just (Bake 16))

```

Вопрос 2. Написать функцию с типом `funC :: Either a b -> (a -> b) -> b` и тестовый случай.

Ответ:

```
funC :: Either a b -> (a -> b) -> b
funC (Left a)  f = f a
funC (Right b) f = b

testF = funC (Right 3) (+1)

```

3.2 Практическая работа 2

Устные вопросы.

Вопрос: Что такое полугруппа (Semigroup)?

Ответ: Тип с ассоциативной бинарной операцией `(<>)`. Единственный закон: `(x <> y) <> z = x <> (y <> z)`.

Вопрос: Что такое моноид (Monoid)?

Ответ: Полугруппа с нейтральным элементом `mempty`. Дополнительные законы: `mempty <> x = x` и `x <> mempty = x`.

Вопрос 1. Каков тип выражения `a = ((\x y z -> x + y + z) <$> Right 3)`?

Ответ: `Either a (Int -> Int -> Int)`

Вопрос 2. Заполнить пропуски так, чтобы выражение `a = (... <*> ... <*> ...)` имело тип `Maybe String`.

Ответ:

```
a = Just (:) <*> Just 'x' <*> Just "ab"

```

3.3 Практическая работа 3

Вопрос. Написать пример монадического связывания `(>>= )` для монады `Writer`.

Ответ:

```
w = writer ("a", [1,2]) >>= \x -> writer (x ++ "b", [3,4])

```

3.4 Практическая работа 4

Устный вопрос. Почему `runGame` имеет тип `Game () -> Maze -> IO ()`, если `Game a` параметризован типом `a`?

Ответ: Тип `Game a` параметризован, чтобы позволить различным вычислениям возвращать разные типы — например, `gameTurn :: Game Bool` возвращает `Bool`, `getNeighbors :: Game [RoomName]` возвращает список комнат. В случае `runGame` передаётся `Game ()`, потому что `gameLoop :: Game ()` не возвращает содержательного значения — он существует только ради эффектов: изменения состояния (`StateT`), накопления лога (`WriterT`) и ввода-вывода

(`IO`). Тип `()` — стандартный способ выразить «нет возвращаемого значения» в Haskell, аналогично `void` в других языках. При раскрутке стека `runMyStateT`, `runWriterT`, `runReaderT` все обёртки снимаются и в итоге остаётся `IO ()`.

3.5 Практическая работа 5

4 Структура репозитория

Репозиторий доступен по адресу: https://github.com/Ali-sbp/Anari-FP-Practice. Корневая директория содержит по одной папке на каждую практическую работу, а также директорию самостоятельной практики.

- `practice 1/` — файлы ПР1.
  - `pr01_1.hs` — функции для работы со списками и кортежами.
  - `pr01_2.hs` — свёртки, развёртки, алгебраические типы данных, выпечка тортов.
- `practice2/` — файлы ПР2.
  - `pr02_1.hs` — классы типов, инстансы.
  - `pr02_2.hs` — деревья, `MyMaybe`, `MyEither`.
  - `pr02_3.hs` — вспомогательный файл.
  - `MyEvolProject/` — cabal-проект: модуль `MyEvolModule.hs` с инстансами классов типов.
  - `myParser/` — stack-проект: `src/MyTypes/` (`MyMaybe`, `MyEither`, `MyTree`), `src/MyParsers/` (`MyParser`, `MyParsec`, `MyAttoparsec`).
- `practice3/` — файлы ПР3.
  - `pr03_1.hs`, `pr03_2.hs` — основные файлы задания.
  - `glitches/` — stack-проект: глитч-арт через манипуляцию байтами (`src/Purefuncs.hs`, `app/Main.hs`).
  - `Route/` — stack-проект: поиск пути в лабиринте через `RWS` (`src/MazeSolver.hs`).
- `practice4/` — файлы ПР4.
  - `pr04_1.hs` — основной файл задания.
  - `MazeGame/` — stack-проект: игра в лабиринт на трансформерном стеке. `src/MyTransformerStack` — ручная реализация `ReaderT`/`WriterT`/`StateT`; `src/GameEngine.hs` — игровая логика; `maze.txt`, `maze1.txt` — файлы лабиринтов.
- `practice5/` — файлы ПР5.
  - `pr05_1.hs` — основной файл задания.
  - `prac5/` — stack-проект: `src/Lib.hs` — функции и черновые тесты с заметками; `test/Spec.hs` — финальные property-тесты на QuickCheck.
- `self practice/` — вся самостоятельная практика.
  - `practice2.hs` – `practice13.hs` — последовательные файлы практики по лекционному материалу (парсеры, классы типов, `Alternative` и др.).
  - `practice14`, `practice15` — скомпилированные программы глитч-арта из книги *Get Programming with Haskell*.
  - `MonadPr/` — практика по монадам и трансформерам: отдельные файлы (`MReaderpr.hs`, `MWriterpr.hs`, `MStatepr.hs`, `MTrpr.hs`, `MPTCpr.hs`, `MRWSpr.hs`, `MONADNOTES.hs`), stack-проекты по лекциям 9–11 (`lec09/`, `lec10/`, `lec11/`), материалы книги (`getprgwithHaskell/`) и практический лабиринт (`Maze/`).
  - `QCpr/` — практика по QuickCheck: `palindrome-testing/`, `primes/` (с исправлением ошибки из книги), `prac5/` (черновик ПР5).

A Код программ

PR1/pr01_1.hs

Функции для работы со списками, кортежами и функции высшего порядка.

Листинг 1: pr01_1.hs

```
-------------------------------------
--   1.  1 --
-------------------------------------

module Pr01_1 where

{-

   myFST, mySND, myTHRD   (a,b,c)



       :
-- myHead -  (   )
-- myTail -
-- myTake -   n
-- myDrop -   n
-- myProduct -
-- myZip -       ,
-- myZip3
-- myUnzip -

         :
-- myFilter -       ( :      if-then-else)
-- myMap -
-- myZipWith -
-- myZipWith3 -
-- myAll -
-- myAny -
-- myComposition -    (.)

-}
import Prelude --for zip
import Data.ByteString.Builder.Prim (primMapByteStringFixed)

-- myFST, mySND, myTHRD :

myFST :: (a , b , c ) -> a
myFST (x , y , z) = x

mySND :: (a , b , c) -> b
mySND (x , y , z) = y

myTHRD :: (a, b , c) -> c
myTHRD (x , y , z) = z

--myHead

myHead :: [a] -> a
myHead (x : xs) = x
myHead [] = error " empty list"

{-
```

```
myHead :: [a] -> a
myHead (x : _) = x  : Q1: this also works right?
myHead [] = error " empty list"
-}
--myTail
myTail :: [a] -> [a]
myTail (x : xs) = xs  -- myTail (_ : xs) = xs
myTail []  = error " empty list"

--myTake
myTake :: Int -> [a] -> [a]
myTake 0 _ = [] -- 0 case
myTake _ [] = [] -- empty list case
myTake n (x : xs) = x : myTake (n - 1) xs -- calling it with n-1 on the tail of the first
     list recursively

--myDrop

myDrop :: Int -> [a] -> [a]
myDrop 0 (x : xs)= x : xs
myDrop _ [] = []
myDrop n (x : xs) = myDrop (n - 1) xs

--myProduct
myProduct :: [Int] -> Int
myProduct [] = 1
myProduct (x : xs) = x * myProduct xs -- myProduct (x : xs) = foldr (*) 1 xs extension
    suggested this


--myZip
myZip :: [a] -> [b] -> [(a,b)]
myZip [] xs = []
myZip xs [] = []
myZip (x:xs) (y:ys) = (x,y) : myZip xs ys

--myZip3
myZip3 :: [a] -> [b] -> [c] -> [(a,b,c)]
myZip3 [] _ _ = []
myZip3 _ [] _ = []
myZip3 _ _ [] = []
myZip3 (x:xs) (y:ys) (z:zs) = (x,y,z) : myZip3 xs ys zs

--myUnzip

myUnzip :: [(a,b)] -> ([a] , [b])
myUnzip [] = ([] , [])
myUnzip((a , b) : rest) =
    let (as , bs) = myUnzip rest
    in (a : as, b: bs)

-- second part :
--myFilter (1)
myFilter :: (a -> Bool) -> [a] -> [a]
myFilter _ [] = []
myFilter f (x : xs)
    | f x == True = x : myFilter f xs  -- syntax note from extension : f x == true :: f x
    | f x == False = myFilter f xs     -- f x == false :: (f x)

--myFilter (2)
```

```
myFilter2 :: (a -> Bool) -> [a] -> [a]
myFilter2 _ [] = []
myFilter2 f (x : xs)=
    if f x
        then x : myFilter2 f xs
    else
        myFilter2 f xs

--myMap

myMap :: (a -> b) -> [a] -> [b]
myMap _ [] = []
myMap f (x :xs) = f x : myMap f xs

--myzipWith

myzipWith :: (a-> b -> c) -> [a] -> [b] -> [c]
myzipWith f [] _ = []
myzipWith f _ [] = []
myzipWith f (x : xs) (y:ys) = f x y : myzipWith f xs ys

--myZipWith3

myZipWith3 :: (a -> b -> c-> d) -> [a] -> [b] -> [c] -> [d]
myZipWith3 f [] _ _ = []
myZipWith3 f _ [] _ = []
myZipWith3 f _ _ [] = []
myZipWith3 f (x : xs) (y : ys) (z : zs) = f x y z : myZipWith3 f xs ys zs

--test case :
testzip3 = myZipWith3 (\a b c -> a+b+c) [1,2,3] [4,5,6][7,8,9]

--myAll :
myAll :: (a -> Bool) -> [a] -> Bool
--myAll _ [] = error "empty list"
myAll f [] = True
myAll f (x : xs)
    | f x = myAll f xs
    | not (f x) = False -- can use otherwise here also ?

myAll2 :: (a -> Bool) -> [a] -> Bool
--myAll2 _ [] = error "empty list"
myAll2 f [] = True
myAll2 f (x : xs) =
    if f x
        then myAll2 f xs
    else
        False

--myAny

myAny :: (a -> Bool) -> [a] -> Bool
myAny f [] = False
myAny f (x : xs)
    | not (f x) = myAny f xs
    | otherwise = True

myAny2 :: (a -> Bool) -> [a] -> Bool
myAny2 f [] = False
myAny2 f (x : xs) =
```

```
    if not (f x)                -- extension note :  "not (not (f x)) || myAny2 f xs"
    instead of the if else block
        then myAny2 f xs
    else
        True

--myComposition (for functions of 1 arguement type a -> b)

myComposition :: (b -> c) -> (a -> b) -> a -> c
myComposition f g x = f (g x)
```

PR1/pr01_2.hs

Свёртки, развёртки, алгебраические типы данных, выпечка тортов.

Листинг 2: pr01_2.hs

```
-------------------------------------
--   1.  2 --
-------------------------------------

module Pr01_2 where
import GHC.Base (BCO, DoubleBox)
--import Distribution.Simple.LocalBuildInfo (prefixRelativeComponentInstallDirs)
--import Lec04 (Ingredients(BakingPowder))
--import Lec04 (CakeDough(ChocolateCakeDough))



{-

  :
-- myZipSave -
-- myUnzipSave -
-- myReverse -
-- myFoldl1 -       (  )
-- myFoldr1 -       (  )
-- myTakeWhile -
-- mySpan -
-- myMaybe -          (maybe)
-- myMap -   map    MyList
-- myUnFoldr -  (   )

--        :
    --
    --
    --

-}

myZipSave :: [a] -> [b] -> ([(a,b)] , Maybe (Either [a] [b]))
myZipSave [] [] = ([] , Nothing)
myZipSave (x: xs) [] = ([] , Just (Left (x:xs)))
myZipSave [] (y:ys) = ([] , Just (Right (y:ys)))
myZipSave (x:xs) (y:ys) =
    let (pairs , leftover) = myZipSave xs ys
    in ( (x,y): pairs , leftover)

-- question : is this normal that in test , it prints "Just" and "right" ?
--ghci> myZipSave [1,2,3] [4,5,6,7,8]
```

```haskell
-- ([(1,4),(2,5),(3,6)],Just (Right [7,8]))

myUnzipSave :: Maybe (Either [a] [b]) -> [(a,b)] -> ([a] , [b])
myUnzipSave Nothing  [] = ([],[])
myUnzipSave Nothing ((a,b) : rest) =
    let (as , bs ) = myUnzipSave Nothing rest
    in (a: as , b : bs )
myUnzipSave (Just (Left (x:xs))) [] = ((x:xs), [])
myUnzipSave (Just (Right (y:ys))) [] = ([] , (y:ys))
myUnzipSave (Just (Left (x:xs))) ((a,b) : rest) =
    let (as , bs) = myUnzipSave (Just (Left (x:xs))) rest
    in ( a: as , b : bs )
myUnzipSave (Just (Right (y:ys))) ((a,b): rest) =
    let (as , bs) = myUnzipSave (Just (Right (y:ys))) rest
    in (a : as , b : bs )

--this looks really stupid , the implementation, should it be this way?

myFoldl :: (b -> a -> b) -> b -> [a] -> b
myFoldl f b [] = b
myFoldl f b (x:xs) = myFoldl f (b `f` x) xs

myReverse :: [a] -> [a]
myReverse xs = myFoldl (\f x -> x : f) [] xs

--myReverse = myFoldl (\f x -> x : f) [] eta reduction suggested by extention?


--myFoldl1
myFoldl1 :: (a->a->a) -> [a] -> a
myFoldl1 f [] = error " asd "
--myFoldl1 f [a] = a
myFoldl1 f (x:xs) = jhonny x xs
    where
        jhonny acc []= acc
        jhonny acc (y:ys) = jhonny (f acc y) ys
--TODO1 done
myFoldl12 :: (a -> a -> a) -> [a] -> a
myFoldl12 f []       = error "empty list"
myFoldl12 f [x]      = x
myFoldl12 f (x:y:ys) = myFoldl12 f (f x y : ys)


--myFoldr , wasted 2 hours and ended up not using it in the fucking implementation!
myFoldr :: (a -> b -> b) -> b -> [a] -> b
myFoldr f b [] = b
myFoldr f b (x:xs) = x `f` (myFoldr f b xs)
--myFoldr1

myFoldr1 :: (a -> a -> a) -> [a] -> Maybe a
myFoldr1 f [] = Nothing
myFoldr1 f [a] = Just a
myFoldr1 f (x : xs) =
    let Just z = myFoldr1 f xs
    in Just (f x z)


--myTakeWhile without folds

mytakeWhile :: (a -> Bool) -> [a] -> [a]
```

```
mytakeWhile p [] = []
mytakeWhile p (x:xs)
    | p x = x : mytakeWhile p xs
    | otherwise = []

--mytakeWhile with folds

mytakeWhile2 :: (a -> Bool) -> [a] -> [a]

--mytakeWhile2 p [] = []
mytakeWhile2 p list = foldr localF [] list
    where
        localF x rest
            | p x = x : rest
            | otherwise = []

--mySpan

mySpan :: (a-> Bool) -> [a] -> ( [a], [a])
mySpan p xs = foldr localF ([],[]) xs
    where
        localF z (xs , ys)
            |p z = (z:xs , ys)
            |otherwise =  ([] , z: xs ++ ys)


--myMaybe

myMaybe :: b -> (a -> b) -> Maybe a -> b
myMaybe n f Nothing = n
myMaybe n f (Just v) = f v


--myMap
data MyList a = MyEmpty | MyCons a (MyList a) deriving Show

myMap :: (a->b) -> MyList a -> MyList b
myMap _ MyEmpty = MyEmpty
myMap f (MyCons x xs) = MyCons (f x) (myMap f xs)

asd= myMap (+1) (MyCons 1 (MyCons 2 (MyCons 3 MyEmpty)))

--myUnfoldr

myUnfoldr :: (b -> Maybe (a , b)) -> b -> [a]
myUnfoldr f b = myMaybe [] (\(x, next) -> x : myUnfoldr f next) (f b)

--random test case
asd2 = myUnfoldr (\n -> if n == 0 then Nothing else Just (n, n-1)) 5


--types
--random practice
data Drink = Tea | Coffee | Juice

drinkTemp :: Drink -> String
drinkTemp Tea = "hot"
drinkTemp Coffee = "hot"
drinkTemp Juice = "cold"

```

```
data Planet = Saturn | Jupiter | Earth
hasrings :: Planet -> Bool
hasrings Saturn= True
hasrings Jupiter = True
hasrings Earth= False


data Shape = Circle Double | Rect Double Double

perimeter :: Shape -> Double
perimeter (Circle r) = 3.14 * 2 * r
perimeter (Rect w h) = w*2 + h*2

--pizza :
data Base = Thinbase | ThickBase deriving Show
data Sause = Tomatosause | Creamsause deriving Show
data Pizza = Margarita | Pepperoni | Vegge deriving Show

makePizza :: Maybe Base -> Maybe Sause -> Maybe Pizza
makePizza (Just Thinbase) (Just Tomatosause) = Just Margarita
makePizza (Just ThickBase) (Just Tomatosause) = Just Pepperoni
makePizza (Just ThickBase) (Just Creamsause) = Just Vegge
-- makePizza (Just Thinbase) (Just Creamsause) = Nothing -- redundant
--how to write anything else = Nothing ?
makePizza _ _ = Nothing

describePizza :: Maybe Pizza -> String
describePizza (Just Vegge) = "thick base + creamsause"
describePizza (Just Margarita) = " Thinbase + tomato sause"
describePizza (Just Pepperoni) = "thicbase + tomatosauce"
describePizza Nothing = "Nothing?"
--how to write any other?
--can use this cause pattern checking is top to bottom!


--cakes
--commenting it for less headache
{-
data Cake = ChocolateCake | BananaCake | CherryCake deriving Show
data Ingredients = Oil | Chocolate | Egg | Flour | Sugar | Bakingpowder | Banana | Cherry
     deriving Show
data Fillingmix = Oilchocolatemix | Bananmix | Cherrymix deriving Show
data Dough = CakeDough deriving Show
data Cakedough = ChocolateDough | BananaDough | CherryDough deriving Show


-- ,
makeCakeMix ::  Ingredients ->  Ingredients ->  Fillingmix
makeCakeMix  Oil Chocolate = Oilchocolatemix
makeCakeMix Chocolate Oil = Oilchocolatemix
-- ...
makeCakeMix Banana Oil = Bananmix
makeCakeMix Oil Banana = Bananmix
makeCakeMix Cherry Sugar = Cherrymix
makeCakeMix Sugar Cherry = Cherrymix

cakeDough :: Ingredients -> Ingredients -> Ingredients -> Ingredients -> Dough
cakeDough Egg Flour Sugar Bakingpowder = CakeDough
-- ...

```

```
--i will make a new one
{-
chocolateCakeDough :: Dough -> Fillingmix -> Cakedough
chocolateCakeDough CakeDough Oilchocolatemix = ChocolateDough
-- ...-}
cakeDoughFinal :: Dough -> Fillingmix -> Cakedough
cakeDoughFinal CakeDough Oilchocolatemix = ChocolateDough
cakeDoughFinal CakeDough Bananmix = BananaDough
cakeDoughFinal CakeDough Cherrymix = CherryDough

--will also make a new one:
--Also error on not caps on "c" ? it will make it a variable wwhich will match anything?
{-
chocolateCake :: Cakedough -> Action -> Cake
chocolateCake ***c***hocolateCakeDough Bake = ChocolateCake
-- ... -}
data Action = Bake deriving Show
makeCake :: Cakedough -> Action ->Cake
makeCake ChocolateDough Bake = ChocolateCake
makeCake BananaDough Bake = BananaCake
makeCake CherryDough Bake = CherryCake
-}
--examples
data Stock = Apples Int | Milk Double deriving Show
useApples :: Int -> Stock -> Maybe Stock
useApples m (Apples n)
    | n > m = Just (Apples (n-m))
    |otherwise = Nothing

useApples _ _ = Nothing

data Ingredients = Oil Double
                |Chocolate Double
                |Egg Int
                |Flour Double
                |Sugar Double
                |Bakingpowder Double
                |Banana Int
                |Cherry Int
                deriving Show

data Cake = ChocolateCake | BananaCake | CherryCake deriving Show
data Fillingmix = Oilchocolatemix | Bananamix | Cherrymix deriving Show
data Dough = CakeDough deriving Show
data Cakedough = ChocolateDough | BananaDough | CherryDough deriving Show
data Action = Bake Int deriving Show
-- ,
makeCakeMix ::  Maybe Ingredients ->  Maybe Ingredients ->  Maybe Fillingmix
makeCakeMix  (Just (Oil n)) (Just (Chocolate m ))
            | n>100 && n<150 && m > 200 && 250>m= Just Oilchocolatemix
makeCakeMix  (Just (Chocolate n)) (Just (Oil m ))
            | m>100 && m<150 && n > 200 && 250>n= Just Oilchocolatemix

makeCakeMix  (Just (Banana n)) (Just (Oil m ))
            | n>2 && n<5 && m > 100 && 150>m= Just Bananamix
makeCakeMix  (Just (Oil n)) (Just (Banana m ))
            | n>100 && n<150 && m > 2 && 5>m= Just Bananamix

makeCakeMix  (Just (Cherry n)) (Just (Sugar m ))
            | n>5 && n<20 && m > 100 && 150>m= Just Cherrymix
```

```
makeCakeMix  (Just (Sugar n)) (Just (Cherry m ))
            | n>100 && n<150 && m > 5 && 20>m= Just Cherrymix
makeCakeMix _ _ = Nothing


cakeDough :: Maybe Ingredients -> Maybe Ingredients -> Maybe Ingredients -> Maybe Ingredients -> Maybe Dough
cakeDough (Just (Egg n)) (Just (Flour m)) (Just (Sugar s)) (Just (Bakingpowder b))
    | n > 2 && n < 4 && m > 100 && m < 150 && s > 200 && s < 250 && b >50 && b <75 = Just CakeDough


cakeDough _ _ _ _ = Nothing

cakeDoughFinal :: Maybe Dough -> Maybe Fillingmix -> Maybe Cakedough
cakeDoughFinal (Just CakeDough) (Just Oilchocolatemix) = Just ChocolateDough
cakeDoughFinal (Just CakeDough)(Just Cherrymix) = Just CherryDough
cakeDoughFinal (Just CakeDough) (Just Bananamix) = Just BananaDough
cakeDoughFinal _ _ = Nothing

makeCake :: Maybe Cakedough -> Maybe Action -> Maybe Cake
makeCake (Just ChocolateDough) (Just (Bake t))
    |t > 15 && t < 20 = Just ChocolateCake
makeCake (Just BananaDough) (Just (Bake t))
    |t > 15 && t < 20 = Just BananaCake
makeCake (Just CherryDough) (Just (Bake t))
    |t > 15 && t < 20 = Just CherryCake
makeCake _ _ = Nothing


testmix = makeCakeMix  (Just (Sugar 101)) (Just (Cherry 10))
testdough = cakeDough (Just (Egg 3))(Just (Flour 101))(Just (Sugar 201))(Just (Bakingpowder 51))
testfinaldough = cakeDoughFinal (Just CakeDough)(Just Cherrymix)

testmake = makeCake(Just CherryDough) (Just (Bake 16))

funC :: Either a b -> (a -> b) -> b
funC (Left a) f = f a
funC (Right b) f= b

testF = funC (Right 3) (+1)
```

PR2/MyEvolProject/MyEvolModule.hs

Ручная реализация инстансов классов типов для типа-суммы `MyEvolution`.

Листинг 3: `MyEvolModule.hs`

```
module MyEvolModule where

--import Data.List(sort)

data MyEvolution =
    LUCA
    | Cyanobacteria
    | Trilobite
    | Ichthyostega
    | Dimetrodon
    | Archaeopteryx
```

```
    | Morganucodon
    | Purgatorius
    | Australopithecine
    | Humans

instance Show MyEvolution where
    show LUCA = "Last Universal Common Ancestor"
    show Cyanobacteria = "Synechococcus"
    show Trilobite ="Paradoxides"
    show Ichthyostega = "Ichthyostega"
    show Dimetrodon = "Dimetrodon"
    show Archaeopteryx = "Archaeopteryx"
    show Morganucodon = "Morganucodon"
    show Purgatorius = "Purgatorius"
    show Australopithecine = "Australopithecus Afarensis"
    show Humans = "Homo Sapiens"

instance Eq MyEvolution where
    LUCA == LUCA = True
    Cyanobacteria == Cyanobacteria = True
    Trilobite == Trilobite = True
    Ichthyostega == Ichthyostega = True
    Dimetrodon == Dimetrodon = True
    Archaeopteryx == Archaeopteryx = True
    Morganucodon == Morganucodon = True
    Purgatorius == Purgatorius = True
    Australopithecine == Australopithecine = True
    Humans == Humans = True
    _ == _ = False

instance Ord MyEvolution where
    compare LUCA LUCA = EQ
    compare LUCA _ = LT
    compare _ LUCA = GT
    compare Cyanobacteria Cyanobacteria = EQ
    compare Cyanobacteria _ = LT
    compare _ Cyanobacteria = GT
    compare Trilobite Trilobite = EQ
    compare Trilobite _ = LT
    compare _ Trilobite = GT
    compare Ichthyostega Ichthyostega = EQ
    compare Ichthyostega _ = LT
    compare _ Ichthyostega = GT
    compare Dimetrodon Dimetrodon = EQ
    compare Dimetrodon _ = LT
    compare _ Dimetrodon = GT
    compare Archaeopteryx Archaeopteryx = EQ
    compare Archaeopteryx _ = LT
    compare _ Archaeopteryx = GT
    compare Morganucodon Morganucodon = EQ
    compare Morganucodon _ = LT
    compare _ Morganucodon = GT
    compare Purgatorius Purgatorius = EQ
    compare Purgatorius _ = LT
    compare _ Purgatorius = GT
    compare Australopithecine Australopithecine = EQ
    compare Australopithecine _ = LT
    compare _ Australopithecine = GT
    compare Humans Humans = EQ
    --compare x y = compare (fromEnum x) (fromEnum y) why didn't we do this ... :((
```

```haskell

instance Enum MyEvolution where
    fromEnum LUCA                = 0
    fromEnum Cyanobacteria       = 1
    fromEnum Trilobite           = 2
    fromEnum Ichthyostega        = 3
    fromEnum Dimetrodon          = 4
    fromEnum Archaeopteryx       = 5
    fromEnum Morganucodon        = 6
    fromEnum Purgatorius         = 7
    fromEnum Australopithecine   = 8
    fromEnum Humans              = 9
    toEnum 0 = LUCA
    toEnum 1 = Cyanobacteria
    toEnum 2 = Trilobite
    toEnum 3 = Ichthyostega
    toEnum 4 = Dimetrodon
    toEnum 5 = Archaeopteryx
    toEnum 6 = Morganucodon
    toEnum 7 = Purgatorius
    toEnum 8 = Australopithecine
    toEnum 9 = Humans
    toEnum _ = error "MyEvolution.toEnum: out of range"
instance Bounded MyEvolution where
    minBound = LUCA
    maxBound = Humans

instance Read MyEvolution where
    readsPrec _ str = case str of
        'L':'a':'s':'t':' ':'U':'n':'i':'v':'e':'r':'s':'a':'l':' ':'C':'o':'m':'m':'o':'n':' ':'A':'n':'c':'e':'s':'t':'o':'r':rest -> [(LUCA, rest)]
        'S':'y':'n':'e':'c':'h':'o':'c':'o':'c':'c':'u':'s':rest -> [(Cyanobacteria, rest)]
        'P':'a':'r':'a':'d':'o':'x':'i':'d':'e':'s':rest -> [(Trilobite, rest)]
        'I':'c':'h':'t':'h':'y':'o':'s':'t':'e':'g':'a':rest -> [(Ichthyostega, rest)]
        'D':'i':'m':'e':'t':'r':'o':'d':'o':'n':rest -> [(Dimetrodon, rest)]
        'A':'r':'c':'h':'a':'e':'o':'p':'t':'e':'r':'y':'x':rest -> [(Archaeopteryx, rest)]
        'M':'o':'r':'g':'a':'n':'u':'c':'o':'d':'o':'n':rest -> [(Morganucodon, rest)]
        'P':'u':'r':'g':'a':'t':'o':'r':'i':'u':'s':rest -> [(Purgatorius, rest)]
        'A':'u':'s':'t':'r':'a':'l':'o':'p':'i':'t':'h':'e':'c':'u':'s':' ':'A':'f':'a':'r':'e':'n':'s':'i':'s':rest -> [(Australopithecine, rest)]
        'H':'o':'m':'o':' ':'S':'a':'p':'i':'e':'n':'s':rest -> [(Humans, rest)]
        _ -> []

data MyEvolution' = LUCA'
                  | Cyanobacteria'
                  | Trilobite'
                  | Ichthyostega'
                  | Dimetrodon'
                  | Archaeopteryx'
                  | Morganucodon'
                  | Purgatorius'
                  | Australopithecine'
```

```
                    | Humans'
                    deriving (Show, Read, Eq, Ord, Enum, Bounded)
```

PR2/myParser/Main.hs

Точка входа проекта `myParser`: вызов трёх реализаций парсера.

Листинг 4: `Main.hs` (myParser)

```
module Main (main) where

import MyTypes.MyTree
import MyTypes.MyMaybe
import MyTypes.MyEither
import MyParsers.MyParser
import MyParsers.MyParsec (plusOrMultParsec, runParser')
import MyParsers.MyAttoparsec(plusOrMultAttoparsec, runParser'')

main :: IO ()
main = do
    --some random tests
  -- MyTree
  let t = Node 1 (Leaf 2) (Node 3 (Leaf 4) (Leaf 5))
  putStrLn (show (fmap (*2) t))
  putStrLn (show (foldr (+) 0 t))

  -- MyMaybe
  putStrLn (show (MyJust "Hello" <> MyJust " World"))
  putStrLn (show (MyJust (+3) <*> MyJust 7))

  -- MyEither
  putStrLn (show (fmap (*2) (MyRight 21) :: MyEither String Int))
  putStrLn (show (MyLeft "err1" <> MyLeft "err2" :: MyEither String String))

  --MyParser
  putStrLn "MyParser:"
  putStrLn $ show (runParser plusOrMult "12*345dsf")
  putStrLn $ show (runParser plusOrMult "12+345dsf")

  putStrLn "Parsec:"
  putStrLn $ show (runParser' plusOrMultParsec "12*345dsf")
  putStrLn $ show (runParser' plusOrMultParsec "12+345dsf")

  putStrLn "Attoparsec:"
  putStrLn $ show (runParser'' plusOrMultAttoparsec "12*345dsf")
  putStrLn $ show (runParser'' plusOrMultAttoparsec "12+345dsf")
```

PR2/myParser/MyTypes/MyMaybe.hs

Пользовательский тип `MyMaybe` с инстансами `Foldable`, `Semigroup`, `Monoid`, `Functor`, `Applicative`.

Листинг 5: `MyMaybe.hs`

```
module MyTypes.MyMaybe where
import Data.Functor as FO
import Data.Monoid as MN


data MyMaybe a = MyNothing | MyJust a  deriving (Show,Eq,Ord)

```

```haskell
instance Foldable MyMaybe where
    foldr _ x MyNothing = x
    foldr f x (MyJust y) = f y x
    --foldl f x (MyJust y) = f x y ??? just out of curiocity
    --foldMap f x =
tst1 = foldr (+) 0 (MyJust 5)
tst2 = foldr (+) 0 MyNothing
tst3 = foldMap show (MyJust 42)

instance Semigroup a => Semigroup (MyMaybe a) where

    -- MyNothing <> _ = MyNothing at first I made them this way but this breaks the monoid law.
    -- _ <> MyNothing = MyNothing
    MyNothing <> x = x
    x <> MyNothing = x
    (MyJust x) <> (MyJust y) = MyJust (x <> y)

tst4 = MyJust "hi" <> MyJust "!"
tst5 = MyNothing <> MyJust [1,2]
tst6 = MyJust [1,2] <> MyJust [3]

instance Semigroup a => Monoid (MyMaybe a) where
    mempty = MyNothing

tst7 = (mempty :: MyMaybe [Int])
tst8 = mappend (MyJust "a") (MyJust "b")
tst9 = mconcat [MyJust "x", MyJust "y", MyNothing]

instance Functor MyMaybe where
    fmap f MyNothing = MyNothing
    fmap f (MyJust a) = MyJust (f a)

tst10 = fmap (*2) (MyJust 5)
tst11 = fmap (*2) MyNothing
tst12 = fmap show (MyJust 42)

instance Applicative MyMaybe where
    pure x = MyJust x
    MyJust f <*> MyJust x = MyJust (f x)
    _ <*> MyNothing = MyNothing
    MyNothing <*> _ = MyNothing

tst13 = (pure 5 :: MyMaybe Int)
tst14 = MyJust (+1) <*> MyJust 5
tst15 = MyNothing <*> MyJust 5

{-
-- Foldable: fold, foldMap, foldr
ghci> fold (MyJust [1,2,3])
[1,2,3]
ghci> fold MyNothing :: [Int]
[]
ghci> foldMap show (MyJust 42)
"42"
ghci> foldMap show MyNothing
""
ghci> foldr (+) 0 (MyJust 5)
5
ghci> foldr (+) 0 MyNothing
```

```
0
ghci> getSum $ foldMap (Sum . (+1)) [1,2,3]
9

-- Semigroup: (<>), sconcat, stimes
ghci> MyJust "hi" <> MyJust "!"
MyJust "hi!"
ghci> MyNothing <> MyJust [1,2] again wthe wrong version , fr last implementation! breaks
     monoid Rule
MyNothing
ghci> MyNothing <> MyJust [1,2] corrected version
MyJust [1,2]
ghci> sconcat (MyJust "a" :| [MyJust "b", MyJust "c"])
MyJust "abc"
ghci> stimes 3 (MyJust "ha")
MyJust "hahaha"

-- Monoid: mappend, mconcat
ghci> mappend (MyJust "a") (MyJust "b")
MyJust "ab"
ghci> mappend MyNothing (MyJust "b") THIS WAS THE FIRST IMPLEMENTATION WHICH BREAKS
    MONOID LAWW !!!!!
MyNothing
ghci> mempty <> MyJust "asd" corrected version
MyJust "asd"
ghci> mconcat [MyJust "a", MyJust "b", MyNothing] SAME HERE !!!
MyNothing
ghci> mconcat [MyJust "a", MyJust "b", MyNothing] corrected version
MyJust "ab"
ghci> mconcat [MyJust [1], MyJust [2], MyJust [3]]
MyJust [1,2,3]

-- Functor: fmap, (<$)
ghci> fmap (*2) (MyJust 5)
MyJust 10
ghci> fmap (*2) MyNothing
MyNothing
ghci> 'x' <$ MyJust 5
MyJust 'x'
ghci> 'x' <$ MyNothing
MyNothing

-- Applicative: pure, (<*>), liftA2, (*>), (<*)
ghci> pure 5 :: MyMaybe Int
MyJust 5
ghci> MyJust (+1) <*> MyJust 5
MyJust 6
ghci> MyNothing <*> MyJust 5
MyNothing
ghci> liftA2 (+) (MyJust 3) (MyJust 4)
MyJust 7
ghci> liftA2 (+) MyNothing (MyJust 4)
MyNothing
ghci> MyJust 1 *> MyJust 2
MyJust 2
ghci> MyJust 1 <* MyJust 2
MyJust 1
-}
```

PR2/myParser/MyTypes/MyEither.hs

Пользовательский тип `MyEither` с инстансами `Foldable`, `Semigroup`, `Functor`, `Applicative`.

Листинг 6: `MyEither.hs`

```haskell
module MyTypes.MyEither where

import Data.Foldable as F()
import Data.Semigroup as S()


data MyEither e a = MyLeft e | MyRight a deriving Show

instance Foldable (MyEither e) where
    foldr _ x (MyLeft _) = x
    foldr f x (MyRight y) = f y x
    --foldMap _ (MyLeft _) = mempty
    --foldMap f (MyRight y) = f y

tst1 :: Integer --wtf is this annoying warninggggggg
tst1 = foldr (+) 0 (MyRight 5)
tst2 = foldr (+) 0 (MyLeft "err")
tst3 = foldMap show (MyRight 42)

instance Semigroup a => Semigroup (MyEither e a) where
    MyLeft x <> _ = MyLeft x
    _ <> MyLeft y = MyLeft y
    MyRight x <> MyRight y = MyRight (x <> y)

tst4 = MyLeft "err" <> MyRight "b"
tst5 = MyRight "a" <> MyRight "b"
tst6 = MyRight [1,2] <> MyRight [3,4]

instance Functor (MyEither e) where
    fmap _ (MyLeft x) = MyLeft x
    fmap f (MyRight x) = MyRight (f x)

tst7 = fmap (+1) (MyRight 5)
tst8 = fmap (+1) (MyLeft "err")
tst9 = fmap show (MyRight 42)

instance Applicative (MyEither e) where
    pure x = MyRight x
    MyLeft e <*> _ = MyLeft e
    _ <*> MyLeft e = MyLeft e
    MyRight f <*> MyRight x = MyRight(f x)

tst10 = (pure 5 :: MyEither String Int)
tst11 = MyRight (+1) <*> MyRight 5
tst12 = (MyLeft "err" :: MyEither String (Int->Int)) <*> MyRight 5

{-
-- Foldable: fold, foldMap, foldr
ghci> fold (MyRight [1,2,3])
[1,2,3]
ghci> fold (MyLeft "err") :: [Int]
[]
ghci> foldMap show (MyRight 42)
"42"
ghci> foldMap show (MyLeft "err")
""
```

```
ghci> foldr (+) 0 (MyRight 5)
5
ghci> foldr (+) 0 (MyLeft "err")
0

-- Semigroup: (<>), sconcat, stimes
ghci> MyRight "a" <> MyRight "b"
MyRight "ab"
ghci> MyLeft "err" <> MyRight "b"
MyLeft "err"
ghci> sconcat (MyRight "a" :| [MyRight "b", MyRight "c"])
MyRight "abc"
ghci> stimes 3 (MyRight "ha")
MyRight "hahaha"

-- Functor: fmap, (<$)
ghci> fmap (+1) (MyRight 5)
MyRight 6
ghci> fmap (+1) (MyLeft "err")
MyLeft "err"
ghci> 'x' <$ MyRight 5
MyRight 'x'
ghci> 'x' <$ MyLeft "err"
MyLeft "err"

-- Applicative: pure, (<*>), liftA2, (*>), (<*)
ghci> pure 5 :: MyEither String Int
MyRight 5
ghci> MyRight (+1) <*> MyRight 5
MyRight 6
ghci> MyLeft "err" <*> MyRight 5
MyLeft "err"
ghci> liftA2 (+) (MyRight 3) (MyRight 4)
MyRight 7
ghci> liftA2 (+) (MyLeft "err") (MyRight 4)
MyLeft "err"
ghci> MyRight 1 *> MyRight 2
MyRight 2
ghci> MyRight 1 <* MyRight 2
MyRight 1
-}
```

PR2/myParser/MyTypes/MyTree.hs

Пользовательский тип `MyTree` с инстансами `Foldable`, `Functor`, `Applicative`.

Листинг 7: `MyTree.hs`

```
module MyTypes.MyTree where

import Data.Functor as FO()
import Data.Monoid as M()

data MyTree a = Leaf a | Node a (MyTree a)(MyTree a) deriving (Show,Eq)
instance Functor MyTree where
    fmap f (Leaf a) = Leaf (f a)
    fmap f (Node x y z) = Node (f x) (fmap f y)(fmap f z)

instance Foldable MyTree where
    foldMap f (Leaf a)         = f a
```

```haskell
    foldMap f (Node a left right) = f a <> foldMap f left <> foldMap f right
tstffs1 = fmap (+1) (Leaf 1)
tstffs2= fmap (+2) (Node 4 (Leaf 1) (Node 4 (Leaf 1)(Leaf 3)))
instance Applicative MyTree where
    pure x  = Leaf x
    Leaf f <*>  Leaf x = Leaf (f x)
    Node f g h <*> Node x left right = Node (f x) (g <*> left) (h <*> right)
    Leaf f <*> Node x left right = Node (f x)(fmap f left)(fmap f right)
    Node f g h <*> Leaf x = Node (f x)(g <*> pure x)(h <*> pure x)
tst21 =Leaf (+1) <*> Leaf 5
tst22 = pure 42 :: MyTree Int
tst23 = Leaf (+10) <*> Node 1 (Leaf 2) (Leaf 3)

tst24 = Node (+1) (Leaf (*2)) (Leaf (+10)) <*> Leaf 5

tst25 = Node (+1) (Leaf (*2)) (Leaf (+10)) <*> Leaf 5

tst26= fmap id (Node 1 (Leaf 2) (Leaf 3)) == Node 1 (Leaf 2) (Leaf 3)

{-
-- Functor: fmap, (<$)
ghci> fmap (+1) (Leaf 1)
Leaf 2
ghci> fmap (*2) (Node 3 (Leaf 1) (Leaf 2))
Node 6 (Leaf 2) (Leaf 4)
ghci> 'x' <$ Leaf 1
Leaf 'x'
ghci> True <$ Node 3 (Leaf 1) (Leaf 2)
Node True (Leaf True) (Leaf True)

-- Applicative: pure, (<*>), liftA2, (*>), (<*)
ghci> pure 5 :: MyTree Int
Leaf 5
ghci> Leaf (+1) <*> Leaf 5
Leaf 6
ghci> Node (+1) (Leaf (*2)) (Leaf (+10)) <*> Leaf 5
Node 6 (Leaf 10) (Leaf 15)
ghci> liftA2 (+) (Leaf 3) (Leaf 4)
Leaf 7
ghci> Leaf 1 *> Leaf 2
Leaf 2
ghci> Leaf 1 <* Leaf 2
Leaf 1
-}

---alternate application type : whole function Tree applied to each single leaf of values
    :
data MyTree1 a = Leaf1 a | Node1 a (MyTree1 a)(MyTree1 a) deriving (Show,Eq)
instance Functor MyTree1 where
    fmap f (Leaf1 a) = Leaf1 (f a)
    fmap f (Node1 x y z) = Node1 (f x) (fmap f y)(fmap f z)
tstffs3 = fmap (+1) (Leaf1 1)
tstffs4= fmap (+2) (Node1 4 (Leaf1 1) (Node1 4 (Leaf1 1)(Leaf1 3)))
instance Applicative MyTree1 where
    pure x = Leaf1 x
    Leaf1 f <*> valueTree = fmap f valueTree
    Node1 f leftFuncTree rightFuncTree <*> valueTree =
        Node1 (f (root valueTree))
             (leftFuncTree <*> valueTree)
             (rightFuncTree <*> valueTree)
```

```
        where
            root (Leaf1 value) = value
            root (Node1 value _ _) = value

-- case: Leaf1 func <*> valueTree  ==>  fmap func over the whole tree
-- Leaf1 (*2) <*> Node1 3 (Leaf1 1) (Leaf1 4)  ==>  Node1 6 (Leaf1 2) (Leaf1 8)
tst1_leaf_func = Leaf1 (*2) <*> Node1 3 (Leaf1 1) (Leaf1 4)

-- case: Node1 func leftFuncTree rightFuncTree <*> valueTree
-- each subtree of funcs broadcasts over the entire valueTree
-- Node1 (+1) (Leaf1 (*2)) (Leaf1 (+10)) <*> Node1 5 (Leaf1 3) (Leaf1 7)
--   root:         (+1) applied to root of valueTree (5)        ==>  6
--   leftFuncTree: Leaf1 (*2) <*> entire valueTree              ==>  Node1 10 (Leaf1 6) (Leaf1 14)
--   rightFuncTree: Leaf1 (+10) <*> entire valueTree            ==>  Node1 15 (Leaf1 13) (Leaf1 17)
tst1_node_func = Node1 (+1) (Leaf1 (*2)) (Leaf1 (+10)) <*> Node1 5 (Leaf1 3) (Leaf1 7)

-- Applicative law: pure id <*> t == t
tst1_law_identity = (pure id <*> Node1 1 (Leaf1 2) (Leaf1 3)) == Node1 1 (Leaf1 2) (Leaf1 3)

-- Applicative law: pure f <*> pure x == pure (f x)
tst1_law_homomorphism = (pure (+1) <*> pure 5 :: MyTree1 Int) == pure (6 :: Int)
```

PR2/myParser/MyParsers/MyParser.hs

Парсер на основе `MyMaybe` с инстансами `Functor`, `Applicative`, `Alternative`.

Листинг 8: `MyParser.hs`

```
module MyParsers.MyParser (runParser,charA, satisfy, char, lower, upper, digit, multiplication, lowers, digits, finalMult,finalPlus,plusOrMult) where

import MyTypes.MyMaybe (MyMaybe(..))


import Data.Char(isLower,isUpper,digitToInt,isDigit)
import Control.Applicative(Alternative(..))



newtype Parser tok a = Parser {runParser :: [tok] -> MyMaybe ([tok], a)}

charA :: Parser Char Char
charA = Parser f where
    f (x : xs ) | x =='A' = MyJust(xs , x)
    f               _ = MyNothing

satisfy :: (tok -> Bool) -> Parser tok tok
satisfy p = Parser f where
    f ( x : xs )  | p x = MyJust(xs , x)
    f               _   = MyNothing


char :: Char -> Parser Char Char
char c = satisfy (==c)

lower :: Parser Char Char
lower = satisfy isLower
```

```haskell

upper :: Parser Char Char
upper = satisfy isUpper

digit :: Parser Char Int
digit = digitToInt <$> satisfy isDigit

instance Functor (Parser tok) where
   -- fmap :: (a->b) -> Parser tok a -> Parser tok b
    fmap g (Parser u) = Parser f where
        f xs = case u xs of
            MyNothing -> MyNothing
            MyJust(xs', x) -> MyJust(xs', g x)

instance Applicative (Parser tok) where
   -- pure :: a -> Parser tok a
    pure x = Parser (\asd -> MyJust (asd, x))

   -- (<*>) :: Parser tok a -> Parser tok a -> Parser tok a
    Parser u <*> Parser v = Parser f where
        f xs = case u xs of
            MyNothing -> MyNothing
            MyJust(xs', g) -> case v xs' of
                MyNothing -> MyNothing
                MyJust (xs'', x) -> MyJust(xs'', g x)

instance Alternative (Parser tok) where
   -- empty :: Parser tok a
    empty = Parser (\_ -> MyNothing)

   -- (<|>) :: Parser tok a -> Parser tok a -> Parser tok a
    Parser u <|> Parser v = Parser f where
        f xs = case u xs of
            MyNothing -> v xs
            result -> result

multiplication :: Parser Char Int
multiplication = (*) <$> digit <* char '*' <*> digit

lowers :: Parser Char String
lowers = ((:) <$> lower <*> lowers) <|> pure [] -- or pure "". same thing in this context

digits :: Parser Char Int
digits = foldl(\x d -> x*10 + d) 0 <$> some digit


finalMult :: Parser Char Int
finalMult = (*) <$> digits <* char '*' <*> digits

finalPlus :: Parser Char Int
finalPlus = (+) <$> digits <* char '+' <*> digits

plusOrMult :: Parser Char Int
plusOrMult = finalMult <|> finalPlus


--digits' :: Parser Char Int
--digits' = (\x y-> x*10 + y) <$> digit <*> digits' <|> 0

{-
```

```
some :: f a -> f [a]
some v = (:) <$> v <*> many v

many :: f a -> f [a]  :                one or more  : parse v , then zero-or-more
many v = some v <|> pure []            zero or more : try some, or return p[]

optional :: Alternative f => f a -> f (Maybe a)
optional v = Just <$> v <|> pure Nothing : try v , wrap in Just or return Nothing
-}
```

PR2/myParser/MyParsers/MyParsec.hs

Реализация парсера с использованием библиотеки `Parsec`.

Листинг 9: `MyParsec.hs`

```
module MyParsers.MyParsec (plusOrMultParsec, runParser') where


import Text.Parsec (Parsec, parse, digit, char, many1,try,ParseError,(<|>))
--importing ParseError for runParser helper to make it look clean
import Text.Parsec.String (Parser)  -- Parser = Parsec String () a
import Data.Char (digitToInt)



digits :: Parser  Int
digits = foldl(\x d -> x*10 + d) 0  <$> many1(digitToInt <$>  digit )

multP :: Parser Int
multP = (*) <$> digits <* char '*' <*> digits

plusP :: Parser Int
plusP = (+) <$> digits <* char '+' <*> digits

plusOrMultParsec :: Parser Int
plusOrMultParsec = try multP <|> plusP

--helper to keep the integrity of main
runParser' :: Parser Int -> String -> Either ParseError Int
runParser' p input = parse p "" input


tst1 = runParser' plusOrMultParsec "12*345asd"

{-
ghci> :t parse
parse
  :: Text.Parsec.Prim.Stream s Data.Functor.Identity.Identity t =>
     Parsec s () a
     -> Text.Parsec.Pos.SourceName -> s -> Either ParseError a

     reason why asd is not shown is because parse throws it away! only returns Either
    Error or result
-}
```

PR2/myParser/MyParsers/MyAttoparsec.hs

Реализация парсера с использованием библиотеки `Attoparsec`.

Листинг 10: MyAttoparsec.hs

```haskell
module MyParsers.MyAttoparsec (plusOrMultAttoparsec, runParser'')where

import Data.Attoparsec.Text (Parser, parseOnly, digit, char, many1)
--import Data.Text.Internal
import Control.Applicative ((<|>))
import Data.Char(digitToInt)
import Data.Text (pack)  -- to convert String -> Text for parseOnly




digitsAtto :: Parser Int
digitsAtto = foldl(\x d -> x*10 + d) 0 <$> many1 (digitToInt <$> digit)

multAtto :: Parser Int
multAtto = (*) <$> digitsAtto <* char '*' <*> digitsAtto

plusAtto :: Parser Int
plusAtto = (+) <$> digitsAtto <* char '+' <*> digitsAtto

plusOrMultAttoparsec :: Parser Int
plusOrMultAttoparsec = plusAtto <|> multAtto

tst1= parseOnly plusOrMultAttoparsec (pack "12*345")

--
{-
helper to keep integrity of main
note how I pack the string for parseOnly, am I smart? lol
-}
runParser'' :: Parser Int -> String -> Either String Int
runParser'' p input = parseOnly p (pack input)
{-
ghci> :t parseOnly
parseOnly :: Parser a -> Data.Text.Internal.Text -> Either String a
just like parsec just returns the result and throws away the rest
-}
tst2 = runParser'' plusOrMultAttoparsec "12*345asd"
```

PR3/glitches/Purefuncs.hs

Чистые функции для глитч-арта: замена байта и сортировка участка.

Листинг 11: Purefuncs.hs

```haskell
module Purefuncs(intToChar, intToBC, replaceByte,sortSection
                ) where

import qualified Data.ByteString as B
import qualified Data.ByteString.Char8 as BC
import System.Random (randomRIO)

intToChar :: Int -> Char
intToChar int = toEnum safeInt
    where safeInt = int `mod` 255

intToBC :: Int -> BC.ByteString
intToBC int = BC.pack [intToChar int]
```

```haskell

replaceByte :: Int -> Int -> BC.ByteString -> BC.ByteString
replaceByte loc chV bytes = mconcat [before,newChar,after]
    where
        (before,rest) = BC.splitAt loc bytes
        after = BC.drop 1 rest
        newChar = intToBC chV

sortSection :: Int -> Int -> BC.ByteString -> BC.ByteString
sortSection start size bytes =
    mconcat [before, changed, after]
  where
    (before, rest) = BC.splitAt start bytes
    (target, after) = BC.splitAt size rest
    changed = BC.reverse (BC.sort target)
```

PR3/glitches/Main.hs

Точка входа проекта глитч-арта: случайные замены, сортировки и их комбинация через `foldM`.

Листинг 12: `Main.hs` (glitches)

```haskell
module Main (main) where

import Purefuncs
import qualified Data.ByteString as B
import qualified Data.ByteString.Char8 as BC
import System.Random(randomRIO)
import System.Environment (getArgs)
import Control.Monad(foldM)

main :: IO ()
main = do
    args <- getArgs
    let fileName = head args
    imageFile <- BC.readFile fileName
    glitched <- randomReplaceByte imageFile
    let glitchedFileName = mconcat ["glitched1_",fileName]
    BC.writeFile glitchedFileName glitched
    putStrLn "glitch 1 (random byte sway) is done!"
    glitched <- randomSortSection imageFile
    let glitchedFileName = mconcat ["glitched2_", fileName]
    BC.writeFile glitchedFileName glitched
    putStrLn "glitch 2 (random sort section) is done!"
--combination of glitches as explained in book
    -- glitched1 <- randomReplaceByte imageFile
    -- glitched2 <- randomSortSection glitched1
    -- glitched3 <- randomReplaceByte glitched2
    -- glitched4 <- randomSortSection glitched3
    -- glitched5 <- randomReplaceByte glitched4

    --using FoldM from Monad as said in the book instead
    glitched <- foldM (\bytes func -> func bytes) imageFile
        glitchActions
    let glitchedFileName = mconcat ["glitched_3xRep_2xSort",fileName]
    BC.writeFile glitchedFileName glitched
    putStrLn "glitch 3 (3xRep + 2x Sort) is done!"


--impure functions .
```

```haskell

--glitch 1 random byte swap and onwards
randomReplaceByte :: BC.ByteString -> IO BC.ByteString
randomReplaceByte bytes = do
    let bytesLength = BC.length bytes
    location <- randomRIO (1, bytesLength)
    chV <- randomRIO (0, 255)
    return (replaceByte location chV bytes)

--glitch 2:
-- random sort section,
randomChar :: IO Char
randomChar = do
    randomInt <- randomRIO (0,255)
    return (toEnum randomInt)

randomSortSection :: BC.ByteString -> IO BC.ByteString
randomSortSection bytes = do
    let sectionSize = 25
    let bytesLength = BC.length bytes
    start <- randomRIO (0,bytesLength - sectionSize)
    return (sortSection start sectionSize bytes)

glitchActions :: [BC.ByteString -> IO BC.ByteString]
glitchActions = [randomReplaceByte ,randomSortSection
    ,randomReplaceByte ,randomSortSection
    ,randomReplaceByte]
```

PR3/Route/MazeSolver.hs

Поиск пути в лабиринте через монаду `RWS` с DFS.

Листинг 13: `MazeSolver.hs`

```haskell
module MazeSolver where

import qualified Control.Monad.RWS as R
import qualified Data.Map.Strict as M
import qualified Data.Set as S

-- using Map.Map String [String] O(log n) instead of just a [(Strung, String)] O(n)
-- but the idea for a maze is (String, [String]) : (current room, [accessible rooms])

type Maze = M.Map String [String]


-- using set here for state RWS r w s a , s :: Set.Set String, for the state.

type Solver a = R.RWS Maze [String] (S.Set String) a
--just a sample maze for testing
sampleMaze :: Maze
sampleMaze = M.fromList
    [ ("start",  ["roomA", "roomB"])
    , ("roomA",  ["roomC"])
    , ("roomB",  ["roomC", "roomD"])
    , ("roomC",  ["finish"])
    , ("roomD",  [])
    , ("finish",  [])
    ]

```

```haskell
-- testing with : R.runRWS :: RWS r w s a -> r -> s -> (a, s, w)

getNeighbors :: String -> Solver [String]
getNeighbors room =
    R.ask >>= \maze -> return (M.findWithDefault [] room maze)

visitRoom :: String -> Solver ()
visitRoom room =
    R.get >>= \visited ->
        R.put (S.insert room visited) >> -- using this for put so we throw away the return () part
            R.tell ["Visiting: " ++ room]

-- helper to check if room is visited (to avoid loops in DFS)
isVisited :: String -> Solver Bool
isVisited room =
    R.get >>= \visited ->
        return (S.member room visited)

-- multiple rooms in state
-- R.runRWS (isVisited "roomB") sampleMaze (S.fromList ["roomA"])


-- going with two mutually dependant functions like some and many:
-- tryNeighbors tries each neighbor using dfs
-- dfs calls tryNeighbors
tryNeighbors :: [String] -> Solver [String]
tryNeighbors [] = return []
tryNeighbors (n:ns) =
    dfs n >>= \path ->
        if null path
        then tryNeighbors ns
        else return path


--very simple dfs algorithm using calling tryNeighbors
-- the idea is very close to some and many

dfs :: String -> Solver [String]
dfs room =
    if room == "finish"
    then R.tell ["Reached: finish"] >> return ["finish"]
    else
        isVisited room >>= \seen ->
            if seen
            then return []
            else
                visitRoom room >>
                getNeighbors room >>= \neighbors ->
                    tryNeighbors neighbors >>= \path ->
                        if null path
                        then return []
                        else return (room : path)

-- visitRoom room >>                -- mark visited + log
-- getNeighbors room >>= \neighbors ->   -- get connected rooms
-- tryNeighbors neighbors >>= \path ->   -- try each one
-- if null path then return []           -- dead end
-- else return (room : path)             -- prepend current room to path
```

PR3/Route/Main.hs

Точка входа проекта Route: запуск решателя лабиринта.

Листинг 14: Main.hs (Route)

```
module Main (main) where
import qualified Control.Monad.RWS as R
import qualified Data.Map.Strict as M
import qualified Data.Set as S
import MazeSolver
import qualified Control.Monad as Control

maze1 :: Maze
maze1 = M.fromList
    [ ("start",  ["roomA", "roomB", "roomC"])
    , ("roomA",  ["roomA1"])
    , ("roomB",  ["roomB1"])
    , ("roomC",  ["roomC1"])
    , ("roomD",  [])
    , ("roomB1",  [])
    , ("roomC1",  ["finish"])
    , ("roomA1",  ["roomA2"])
    , ("roomA2",  ["roomA3"])
    , ("roomA3",  ["roomA4"])
    , ("roomA4",  [])
    , ("finish",  [])
    ]

main :: IO ()
main =
    let (path, _, logs) = R.runRWS (dfs "start") maze1 S.empty
    in mapM_ putStrLn logs >>
       putStrLn ("path: " ++ show path)

-- mtl isnt needed, can do with trans only , RWS there is in Control.Monad.Trans.RWS
```

PR4/MazeGame/MyTransformerStack.hs

Ручная реализация ReaderT, WriterT, MyStateT и класса MonadTrans.

Листинг 15: MyTransformerStack.hs

```
{-# LANGUAGE InstanceSigs #-}
module MyTransformerStack where

import Data.Functor.Identity

-- ReaderT copied from my practice file
newtype ReaderT r m a = ReaderT { runReaderT :: r -> m a }

reader :: Monad m => (r -> a) -> ReaderT r m a
reader f = ReaderT $ return . f

instance Functor m => Functor (ReaderT r m) where
    fmap :: (a->b) -> ReaderT r m a -> ReaderT r m b
    fmap f x = ReaderT $ fmap f . runReaderT x

instance Applicative m => Applicative (ReaderT r m) where
    pure :: a -> ReaderT r m a
    pure = ReaderT . const . pure
```

```haskell

    (<*>) :: ReaderT r m (a -> b) -> ReaderT r m a -> ReaderT r m b
    f <*> v = ReaderT $ \e -> runReaderT f e <*> runReaderT v e

instance Monad m => Monad (ReaderT r m) where
    (>>=) :: ReaderT r m a -> (a -> ReaderT r m b) -> ReaderT r m b
    m >>= k = ReaderT $ \e -> do
        v <- runReaderT m e
        runReaderT (k v)e

class MonadTrans t where
    lift :: Monad m => m a -> t m a

instance MonadTrans (ReaderT r) where
    lift :: Monad m => m a -> ReaderT r m a
    lift m = ReaderT $ \_ -> m

ask :: Monad m => ReaderT r m r
ask = ReaderT return

asks :: Monad m => (r -> a) -> ReaderT  r m a
asks f = ReaderT $ return . f

local :: (r -> r) -> ReaderT r m a -> ReaderT r m a
local f x = ReaderT $ runReaderT x . f

-- writerT copied from my practice files

newtype WriterT w m a = WriterT {runWriterT :: m (a, w)}
instance Functor m => Functor (WriterT w m) where
    fmap f = WriterT . fmap update . runWriterT
        where update ~(y, log) = (f y, log)

instance (Monoid w, Applicative m) => Applicative (WriterT w m) where
    pure :: a -> WriterT w m a
    pure x = WriterT $ pure (x, mempty)
    (<*>) :: WriterT w m (a -> b) -> WriterT w m a -> WriterT w m b
    f <*> v = WriterT $ liftA2 update (runWriterT f) (runWriterT v)
        where update ~(g,w) ~(x,w') = (g x, w `mappend` w')

instance (Monoid w, Monad m) => Monad (WriterT w m) where
    m >>= k = WriterT $ do
        ~(v, w) <- runWriterT m
        ~(v', w') <- runWriterT (k v)
        return (v' , w <> w')

instance Monoid w => MonadTrans (WriterT w) where
    lift :: Monad m => m a -> WriterT w m a
    lift m = WriterT $ do
        x <- m
        return (x, mempty)

listen :: Monad m => WriterT w m a -> WriterT w m (a, w)
listen m = WriterT $ do
    ~(a,w ) <- runWriterT m
    return ((a,w), w)

censor :: Monad m => (w->w) -> WriterT w m a -> WriterT w m a
censor f m = WriterT $ do
    ~(a,w ) <- runWriterT m
```

```
    return (a , f w)

tell :: Monad m => w -> WriterT w m ()
tell w = writer ((), w)
writer :: Monad m => (a,w) -> WriterT w m a
writer = WriterT . return

-- type Writer w a = WriterT w Identitiy a -- now I get it LOL

--StateT :

newtype MyStateT s m a = MyStateT { runMyStateT :: s -> m (a, s) }

instance Functor m => Functor (MyStateT s m) where
    fmap f st = MyStateT $ \s ->
        fmap (\(a, s') -> (f a, s')) (runMyStateT st s)

instance Monad m => Applicative (MyStateT s m) where
    pure x = MyStateT $ \s -> return (x, s)

    f <*> v = MyStateT $ \s -> do
        (g, s') <- runMyStateT f s
        (x, s'') <- runMyStateT v s'
        return (g x , s'')

instance Monad m => Monad (MyStateT s m) where
    m >>= k = MyStateT $ \s -> do
        (v, s') <- runMyStateT m s
        runMyStateT (k v) s'

instance MonadTrans (MyStateT s) where
    lift m = MyStateT $ \s -> do
        x <- m
        return (x, s)

get :: Monad m => MyStateT s m s
get      = MyStateT $ \s -> return (s, s)

put :: Monad m => s -> MyStateT s m ()
put  s = MyStateT $ \_ -> return ((), s)

modify :: Monad m => (s -> s) -> MyStateT s m ()
modify f = MyStateT $ \s -> return ((), f s)

--tst1 :: (Int, Int)
tst1 = runIdentity $ runMyStateT (do { put 10; modify (*3); x <- get; return x }) 0


-- game sta
```

PR4/MazeGame/GameEngine.hs

Игровая логика на трансформерном стеке: парсер лабиринта, игровой цикл, логирование.

Листинг 16: GameEngine.hs

```
module GameEngine where

import MyTransformerStack
import Data.Maybe(fromMaybe)
```

```haskell
import qualified Data.Map.Strict as M

type RoomName = String
type Maze = M.Map String [String]

-- the transformer stack , wrote it cool xixi xaxa

type Game a = MyStateT RoomName
                (WriterT [String]
                    (ReaderT Maze IO)) a

runGame :: Game () -> Maze -> IO ()
runGame game maze = do
    ((_, _), movelog) <- runReaderT (runWriterT (runMyStateT game "start")) maze
    putStrLn "\n *** Game Over, Your path: ***"
    mapM_ putStrLn movelog
    --writing to logs.txt
    saveLogs movelog



sampleMaze :: Maze
sampleMaze = M.fromList
    [ ("start",   ["hallway"])
    , ("hallway", ["start", "library", "garden"])
    , ("library", ["hallway", "finish"])
    , ("garden",  ["hallway"])
    , ("finish",  [])
    ]

getNeighbors :: RoomName -> Game [RoomName]
getNeighbors room = do
    maze <- lift $ lift ask
    return (M.findWithDefault [] room maze)

-- writing this so that I dont have to write life 100 times
io :: IO a -> Game a
io = lift . lift . lift


--using elem : elem :: (Foldable t, Eq a) => a -> t a -> Bool
-- helper to see possible moves from ~> to
canMoveTo :: RoomName -> RoomName -> Game Bool
canMoveTo from to = do
    neighbors <- getNeighbors from
    return (to `elem` neighbors)

-- one game turn

gameTurn :: Game Bool -- true ~> keep playing, false ~> stop
gameTurn = do
    room <- get
    neighbors <- getNeighbors room
    io $ putStrLn $ "\nYou are in: " ++ room
    io $ putStrLn $ "Neighbors: " ++ show neighbors
    io $ putStr "Move to (or 'quit'): "
    cmd <- io getLine
    case cmd of
        "quit" -> return False
        _      -> do
```

```
            valid <- canMoveTo room cmd
            if not valid
                then do
                    io $ putStrLn "Invalid move!"
                    return True
                else do
                    put cmd
                    lift $ tell ["moved: " ++ room ++ " -> " ++ cmd ]
                    -- 1 lift for tell (WriterT)
                    return (cmd /= "finish")


gameLoop :: Game ()
gameLoop = do
    continue <- gameTurn
    if continue then gameLoop else return ()


-- each line of the file : "roomName neighbor1 neighbor2 ..."
-- for example  "hallway start library garden"
parseMaze :: String -> Maze
parseMaze = M.fromList . map parseLine . lines
  where
    parseLine line = case words line of
        (room:neighbors) -> (room, neighbors)
        []               -> ("",   [])


loadMaze :: FilePath -> IO Maze
loadMaze path = parseMaze <$> readFile path

--adding extra logger to save to file for practice

saveLogs :: [String] -> IO ()
saveLogs movelog = do
    appendFile "logs.txt" "\n=== New Game ===\n"
    appendFile "logs.txt" (unlines movelog)
```

PR4/MazeGame/Main.hs

Точка входа игры в лабиринт.

Листинг 17: `Main.hs` (MazeGame)

```
module Main (main) where

import GameEngine

main :: IO ()
main = do
    maze <- loadMaze "maze.txt"
    runGame gameLoop maze
```

PR5/prac5/Lib.hs

Функции `addMod` и `reverseWords` с черновыми property-тестами.

Листинг 18: `Lib.hs`

```
module Lib
```

```haskell
    ( addMod
    , reverseWords
    ) where

import Test.QuickCheck

addMod :: Int -> Int -> Int -> Int
addMod x y m
    | m <= 0 = error "modulus must be positive"
    | otherwise = (x + y) `mod` m


-- prop1 : the definition. Note : using Positive Int here
--                                to match the function definition guard

-- prop1 :: Int -> Int -> Positive Int -> Bool
-- prop1 x y (Positive m) =
--     addMod x y m === (x + y) `mod` m
-- quickCheck (verbose prop1)
-- ghci> quickCheck prop1
-- +++ OK, passed 100 tests.
-- ghci>

-- prop2 : identity
-- prop2 :: Int -> Positive Int -> Bool
-- prop2 x (Positive m) =
--     addMod x 0 m == x `mod` m

-- ghci> quickCheck prop2
-- +++ OK, passed 100 tests.
-- ghci>

-- prop3 : commutativnost
-- prop3 :: Int -> Int -> Positive Int -> Bool
-- prop3 x y (Positive m) =
--     addMod x y m == addMod y x m

-- ghci> quickCheck prop3
-- +++ OK, passed 100 tests.
-- ghci>

------- reverseWords -------------


reverseWords :: String -> String
reverseWords = unwords . reverse . words

-- prop4 : empty string
-- prop4 = reverseWords "" == ""

-- ghci> quickCheck prop4
-- +++ OK, passed 100 tests.
-- ghci>

--prop5 : one word

-- for this I searched a bit and came across listOf1 :: Gen a -> Gen [a] which generates from given values
-- and after checking the stuff that quickCheck generated (like " a" and "\ndsa" and the other garbage)
```

```
-- i limited the generated data this way in order not to discard 1 Milion tests samples
-- ghci> quickCheck (withMaxSuccess 10000 prop5')
-- +++ OK, passed 10000 tests; 14586 discarded.
-- ghci>
-- so instead of discarding all of this, I am limiting the quickCheck data instead.
-- and overall i dont like how trivial this guard logic is : (&& s == unwords (words s))
-- its legit the same backwards logic of the isPalindrome case from the book where
-- we use is isPunctuation in both the wrapper and the test itself ...
-- prop5' :: String -> Property
-- prop5' s =
--     length (words s) == 1 && s == unwords (words s) ==> reverseWords s == s

-- prop5 :: String -> Property
-- prop5 =
--     forAll (listOf1 (elements ['a'..'z'])) $ \word ->
--         reverseWords word == word


-- prop6 changing order in multi word
-- checking for the reverse order in the list (if i understood the task correctly)
-- using words to limit the garbage generated here.
-- ghci> :t words
-- words :: String -> [String]

-- Prop6 :: String -> Property
-- prop6 s =
--     let ws = words s
--     in length ws > 1 ==> words (reverseWords s) == reverse ws

-- ghci> quickCheck (withMaxSuccess 10000 prop6)
-- +++ OK, passed 10000 tests; 42401 discarded.
-- ghci>

-- equivalant but in String -> Bool
-- prop6 :: String -> Bool
-- prop6 s =
--     let ws = words s
--     in length ws <= 1 || words (reverseWords s) == reverse ws



-- prop7 : double application returns the original string
-- for the current implementation this holds for normalized strings
-- with single spaces between words and no leading/trailing whitespace

-- prop7 :: String -> Property
-- prop7 s =
--     s == unwords (words s) ==> reverseWords (reverseWords s) == s

-- ghci> quickCheck (withMaxSuccess 10000 prop7)
-- +++ OK, passed 10000 tests; 9853 discarded.
-- ghci>

--equivalant but in String -> Bool
-- prop7 :: String -> Bool
-- prop7 s =
--     s /= unwords (words s) || reverseWords (reverseWords s) == s
```

PR5/prac5/Spec.hs

Финальные property-тесты на QuickCheck.

Листинг 19: Spec.hs

```haskell
module Main (main) where

import Lib
import Test.QuickCheck

prop_addModMatchesModuloAddition :: Int -> Int -> Positive Int -> Bool
prop_addModMatchesModuloAddition x y (Positive m) =
    addMod x y m == (x + y) `mod` m

prop_addModHasNeutralElement :: Int -> Positive Int -> Bool
prop_addModHasNeutralElement x (Positive m) =
    addMod x 0 m == x `mod` m

prop_addModIsCommutative :: Int -> Int -> Positive Int -> Bool
prop_addModIsCommutative x y (Positive m) =
    addMod x y m == addMod y x m

prop_reverseWordsEmptyString :: Bool
prop_reverseWordsEmptyString =
    reverseWords "" == ""

prop_reverseWordsSingleWord :: String -> Bool
prop_reverseWordsSingleWord s =
    let ws = words s
    in length ws /= 1 || s /= unwords ws || reverseWords s == s

prop_reverseWordsChangesWordOrder :: String -> Bool
prop_reverseWordsChangesWordOrder s =
    let ws = words s
    in length ws <= 1 || words (reverseWords s) == reverse ws

prop_reverseWordsTwiceReturnsOriginal :: String -> Bool
prop_reverseWordsTwiceReturnsOriginal s =
    s /= unwords (words s) || reverseWords (reverseWords s) == s

main :: IO ()
main = do
    putStrLn "Testing addMod..."
    putStrLn "Checking modular addition matches the mathematical definition..."
    quickCheckWith stdArgs {maxSuccess = 1000} prop_addModMatchesModuloAddition
    putStrLn "Checking 0 is the neutral element for modular addition..."
    quickCheck prop_addModHasNeutralElement
    putStrLn "Checking modular addition is commutative..."
    quickCheck prop_addModIsCommutative
    putStrLn "Testing reverseWords..."
    putStrLn "Checking reversing an empty string returns an empty string..."
    quickCheck prop_reverseWordsEmptyString
    putStrLn "Checking reversing a single word returns the same word..."
    quickCheck prop_reverseWordsSingleWord
    putStrLn "Checking reversing several words changes their order..."
    quickCheck prop_reverseWordsChangesWordOrder
    putStrLn "Checking reversing words twice returns the original normalized string..."
    quickCheck prop_reverseWordsTwiceReturnsOriginal
```